01 Recommended direction

Decision and feasibility

Build a native C++ analysis and arrangement engine, expose a small command interface through Autodesk Function Publishing, and use MAXScript rollouts for the first UI. This matches the requested architecture and provides room for efficient geometry processing without requiring Python, a cloud service, an LLM, or GPU inference at runtime. [S01]

The product should combine three capabilities: geometry determines where an object can fit; categories determine which objects belong on a piece of furniture; recipes determine how those objects form a coherent arrangement. A TV console category can permit books, frames and vases while preserving a television zone and equipment access.

Decision	Recommended starting point
Target furniture	Static TV consoles, open shelves and coffee tables with rigid, predominantly horizontal supports.
Generation	Library-driven native instances, with a saved seed, locked groups and editable region overrides.
Geometry	Support polygons with holes, local overhead clearance, accessibility flags and collision representations.
Visual quality	A small, authored recipe library with deliberate empty space and bounded variation.
Reliability	Main-thread snapshot, worker-only computation, main-thread validation and commit.
Initial platform	One chosen Max release; validate other releases as separate builds.

The idea has substantial prior art. Scatter Tools already documents furniture population, and research systems study support hierarchies and small-object arrangements. The strongest product hypothesis is the integration of dependable detection, reusable categories, composition recipes and artist-controlled regeneration. Its value must be tested against those existing workflows. [S09, S17]

This is a feasibility and architecture report dated 17 October 2026. Three research subagents investigated the SDK, geometry algorithms and existing products; the main agent reconciled their findings and checked asset/integration issues. Vendor claims below are documented capabilities, not independent performance results. No Max plugin was compiled, no product trial was run and no scene benchmark was performed.

02 What is already documented

Existing products and the competitive baseline

Product	Documented capability	Relevance and unresolved detail
Spline Dynamics Scatter Tools [S09 S10]	Auto-populates flat horizontal furniture surfaces; handles groups and collision checks. Redistributes selected existing objects.	Closest verified Max reference. Explicit semantic categories, stored clearance maps and recipe systems are not established by the cited manual.
model+model Bookmanager 2 [S11]	Places books on shelf polygons; horizontal, leaning and flipped poses; seeds, cloning, presets and pivot checks.	A strong reference for book preparation and shelf controls. General accessory styling is outside the cited description.
Chaos Scatter [S12]	Surface and volume distribution, randomized transforms and overlap controls.	Established general scatter behavior. Approximate bounding regions do not establish compartment interpretation.
ForestPack [S13]	Multiple surfaces, projection, filters and refresh controls.	Reference for large-scene distribution infrastructure. Semantic furniture interpretation is unverified here.
RailClone [S14]	Procedural shelf/book construction with randomization and generators.	Useful when shelf structure is known parametrically; arbitrary imported-model interpretation is not demonstrated by that tutorial.
SiNi Disperse [S15]	Artist painting of geometry and proxies.	Reference for local placement, direct editing and artist feedback.

Scatter Tools automatically determines orientation/anchor for its furniture mode and documents checks against both other scattered objects and the distribution geometry. If an object cannot be placed, it remains at its original location. Restricting placement to part of a surface is documented through a cloned/cut distribution object. Its manual does not disclose the underlying collision representation. [S10]

These are evidence boundaries: an undocumented feature is unknown, rather than proven absent. Vendor statements such as collision avoidance still need close-up geometric tests on our own fixtures. The products and manuals were researched, not installed or benchmarked.

The screenshot supplied in the conversation is an idea reference. Its source has not been established, so no attribution to a particular commercial tool is assumed.

03 Ideas to study and limits to retain

Published research that informs the design

HSM, presented at 3DV 2026, is the closest research reference: it extracts horizontal and vertical mesh regions, derives clearance and compartments, and places compositional motifs through a support hierarchy. Its paper reports imperfect region extraction and discusses geometry quality, inaccessible spaces and limited motif coverage. That supports an editable-detection workflow. Its implementation and research results do not establish production robustness on arbitrary Max imports. [S16, S17]

Reference	Useful concept	How it informs this product
SceneMotifCoder [S18]	Recurring arrangements represented by parameterized programs.	Author compact rules for stacks, rows and object-relative groups; vary approved parameters.
SceneSmith [S19]	Hierarchical planning with geometric and physical refinement.	Keep semantic intent separate from exact fitting and final validation.
Make It Home [S20]	Spatial priors and ergonomic layout objectives.	Score functional clearance and layout preferences after hard geometric checks.
Clutterpalette [S21]	Support-aware suggestions for small object categories.	Treat what belongs here as a separate task from whether it fits.
PlanIT [S22]	Relation graph followed by concrete scene instantiation.	Use explicit relationships. Its repository warns that needed models and metadata are unavailable.

Recommended baseline: encode a modest set of recipes directly as validated declarative data consumed by C++. Recipes select prepared assets and permitted poses; a bounded search fits their relationships to a target region. This makes failures explainable and gives artists repeatable results.

An optional future AI layer can suggest tags, interpret a brief such as "minimal styling, books, no plants", or rank approved recipes. The C++ engine should still own support, fitting, collision and commit. AI requests should map to a constrained schema; arbitrary generated scripts are unnecessary for this workflow.

Research code and data have separate permissions. HSM code is MIT licensed, while its dataset and the HSSD model dataset are CC BY-NC 4.0. The code license does not grant rights to bundle those meshes in a commercial accessory library. [S47, S48, S49]

04 Proposed component boundaries

C++ core and MAXScript UI architecture

Component	Responsibility
DressingCore C++ library	Plain-data meshes, region extraction, collision structures, recipes, selection, search, scoring and deterministic results. No Max SDK dependency.
DressingMax native adapter	Evaluated scene access, transforms, units, references, notifications, instancing, viewport overlays and undo participation.
Published command interface	Assign profile; register asset; analyze; start/cancel job; query diagnostics; preview; commit; lock; bake.
MAXScript UI	Node pickers, category/recipe controls, density, seeds, status and command invocation. Keep geometry loops out of the UI.
Scene controller/state	Per-setup settings, native references, region overrides, ownership and generation revision.
Library and presets	Versioned asset metadata and recipe data; editable without recompiling geometric algorithms.

Function Publishing makes C++ interfaces callable from MAXScript. Pass node references and small settings, then return job IDs and compact summaries. Moving large vertex arrays through MAXScript would defeat this boundary. A dedicated MAXScript extension DLL is not needed solely to expose published functions. [S01]

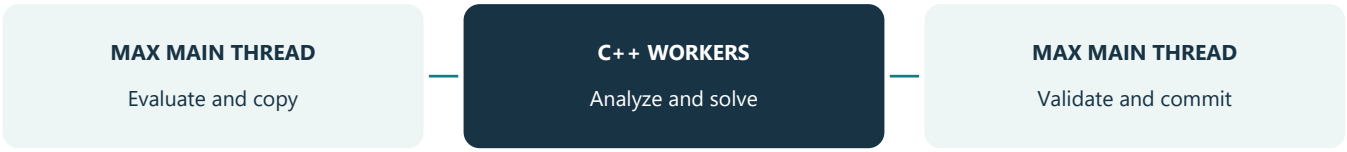
A small GUP can host application-level services, caches and callback lifetime. Per-furniture state belongs on scene-owned objects or attributes. Prototype a C++ custom attribute on a built-in controller node and compare it with a custom helper; settle that choice through save/clone/merge and missing-plugin tests. A GUP alone is a poor owner for every setup. [S52, S33]

Attach instance-specific category overrides, seed and exclusions to the furniture node or setup controller, rather than automatically to its shared base mesh. Two instances of the same cabinet may need different arrangements. Keep reusable asset definitions separate from instance-specific placement state.

MAXScript rollouts are appropriate initially. Native C++ Qt remains an option for a future thumbnail browser or recipe editor; it introduces target-version Qt requirements and should follow a demonstrated UI need. [S42]

05 Threading and transactions

Safe execution and scene lifecycle



Autodesk identifies the reference system and node evaluation system as single threaded. A C++ implementation must not call into them concurrently from geometry workers. Locks around our own data do not make the Max scene API thread safe. [S02]

- Snapshot: on the main thread, evaluate the selected furniture and relevant obstacles at an explicit time, copy supported geometry and transforms, and record unit/settings/source revisions.

- Solve: workers operate on owned vertex/index arrays and value types. Build acceleration structures, extract regions, produce candidate layouts and diagnostics, and check cancellation regularly.

- Commit: return to the main thread, revalidate references and revisions, and reject stale results if the scene changed. Apply only owned output changes as one undoable action.

Do not hold an undo transaction open while solving asynchronously. The UI or command caller should own the short commit transaction; published functions participate in it. Parameter blocks cover their changes, while custom state may need RestoreObj integration. Undo must restore the previous layout and ownership, and redo must restore the same accepted transforms. [S37, S38]

Callbacks should mark caches dirty and queue work. Debounce repeated geometry/transform events and suppress self-triggered regeneration loops. Transform changes, topology edits, asset revisions, units and time have different invalidation scopes. [S39]

On scene load, reconnect and validate state before evaluating geometry. Native references are incomplete during Load(); Autodesk documents post-load ordering constraints. Prefer user-triggered analysis or safe deferred work. Cancel jobs on reset, open and shutdown, and avoid a worker/main-thread waiting cycle. [S36]

06 Main-thread analysis input contract

Reading furniture geometry reliably

Use evaluated geometry at the chosen time so modifier results are represented. Autodesk documents EvalWorldState(), conversion to TriObject when supported, and the rule that a conversion result is deleted only when it is a separately owned temporary. GetObjectRef() alone is not a reliable substitute for the evaluated shape. [S03]

- Resolve the whole furniture assembly: attached mesh, separate boards, nested groups and child nodes. Keep source identities so an artist can diagnose which part produced a region.

- Normalize physical units through GetSystemUnitScale()/GetSystemUnitInfo(). Work close to the furniture origin in a defined local frame, with explicit up/front axes and unit-aware tolerances.

- Apply object offsets, parent transforms and the correct evaluated coordinate convention. Test world-space modifiers explicitly. Do not multiply an already transformed mesh twice.

- Bake nonuniform scale, mirrors and shear into analysis vertices where downstream collision APIs expect rigid transforms. Recompute geometric face normals and winding consistently; shading normals are unsuitable as the sole support test.

Max distinguishes node and object transforms, including object offsets and world-space modifier context. Current units APIs differ from older examples that use GetMasterScale. These details should be covered by small labelled fixture scenes before testing elaborate cabinets. [S04, S05]

Geometry source	Required treatment
Ordinary evaluated mesh	Copy triangles and retain source mapping.
Renderer proxy	Use a verified adapter or an explicitly supplied analysis mesh.
Display box/point cloud	Label as approximate; do not infer shelf openings from it.
USD, XRef, procedural object	Separate compatibility target until its evaluated representation is tested.
Unsupported source	Explain what is missing and allow authored regions/colliders.

V-Ray and Corona document multiple proxy preview modes, including representations that differ greatly from full geometry. Preserve the render asset and its materials; use a separate analysis representation where necessary. [S43, S44]

07 Proposed geometry pipeline

Detecting supports and shelf openings

Represent the furniture as multiple usable surfaces and nearby obstacles. A whole-object bounding box loses the empty spaces; a convex hull around a cabinet closes its openings. A single downward ray sees the highest board and can miss every shelf underneath.

- Diagnose degenerate faces, extreme scales, duplicate geometry, boundaries and inconsistent orientation. Repair only an analysis copy when appropriate; preserve the source model.

- Find approximately planar, near-horizontal patches using gravity, angular tolerance, plane distance and minimum physical area. Grow adjacency within a mesh and merge separate coplanar parts conservatively.

- Project each patch into a 2D surface frame. Union its triangles into boundaries with holes. Retain cutouts and gaps instead of filling them through an enclosing rectangle.

- Query space above the patch, then subtract blocked areas. Reject undersides, tiny ledges and regions that cannot fit any allowed prepared asset. Keep uncertain regions visibly editable.

Plane-region growing is an established geometric technique; CGAL documents region thresholds and planar fitting. Clipper2 provides polygon booleans and offsets suitable for the resulting 2D boundaries. These libraries are building blocks, rather than ready-made furniture understanding. [S23, S26]

Store overhead clearance as a local query or field h(x,y), since a shelf may have a sloped ceiling, supports or protrusions. A single minimum height can be useful for conservative filtering but can reject valid locations; a single maximum can miss collisions. Sampling is only a prefilter, followed by final 3D checks.

A compartment is a useful grouping of connected support areas with nearby vertical boundaries and usable clearance. It need not be a watertight cavity. Mark whether it is exposed, behind a door or otherwise inaccessible; allow a front-facing accessibility override.

Avoid making closed-solid containment the universal prerequisite. CGAL containment has closed-mesh/self-intersection conditions; winding-number methods improve handling of imperfect geometry but retain assumptions. Open imported shells need surface-based reasoning and explicit uncertainty. [S24, S25]

08 Support, containment and collision

Validating a candidate placement

Treat an accessory as three separate representations: visual geometry, supporting contacts and occupied collision volume. For a plant, the pot determines support while the leaves occupy more space. For a tray, the inner surface supports objects while its rim remains an obstacle.

Check	Proposed rule
Eligible object	Furniture category, region role, recipe, pose and physical size must agree.
Footprint	Contact polygon or pads must lie within the support boundary after the requested edge margin, preserving holes.
Clearance	Use local overhead/side filtering, followed by 3D validation of the complete candidate.
Collision	Test furniture, relevant existing objects, locked output and accepted generated objects.
Contact	Allow only intended support contact within tolerance; keep checking other parts of the supporting object.
Stability	For upright rigid poses, use actual contacts and an authored or approximate centre of mass with a safety margin.

Use a spatial broadphase to reject distant objects, then collision and distance queries with suitable simplified shapes. FCL is a candidate library for these queries; Clipper2 can test whether a footprint leaves any area outside the usable region. [S29, S26]

Triangle-surface intersection alone does not prove that solids do not overlap: one object can lie entirely inside another without surfaces intersecting. Closed convex/compound colliders and appropriate containment checks are needed where that case is possible. Excessively coarse colliders can also reject arrangements that would fit.

The entire supporting shelf, tray or book must not be excluded from collision tests. Only the intended contact is permitted; otherwise the solver could push a vase into a shelf side or through a tray rim.

Store stacking as an acyclic support graph, such as shelf to book stack to ornament. Generate and validate parents before children. A change to a stack invalidates its supported ornament. Leaning frames need prepared stable poses or multi-contact logic; upright support rules alone do not validate them.

Physics settling can be evaluated later if it improves visible quality. Bullet is a possible optional dependency. It should not be required for every initial candidate or used to hide an invalid placement search. [S32]

09 What belongs here and how it should look

Furniture categories and arrangement recipes

Naming should suggest categories, while persistent metadata stores the confirmed assignment. A label such as TV_Console_01 can map to TV console. Geometry supplies measurements and support regions; it cannot reliably infer the intended furniture use or decoration style from a name-free mesh alone.

Separate the furniture category from the surface role. A TV console may have an exposed top, low shelf, side niche and equipment area. Each role can allow a different asset set and different recipes. Give the artist a front direction so frames, book spines and reserved TV zones can align meaningfully.

Starter recipe	Relationship and variation
Upright book run	Approved upright poses, bounded gaps and count; optional bookend.
Flat book stack	Prepared flat poses; size/order rules; limited rotation.
Stack with ornament	Ornament supported by the top book region, not merely by stack bounding height.
Tall and low group	Vase or plant plus a lower accessory; controlled gap and height contrast.
Frame and accessory	Prepared frame pose and a small nearby group, aligned to furniture front.
Tray group	Nested arrangement supported inside the tray, with rim clearance.
Console end group	One or two groups with reserved screen/equipment zones and intentional empty space.

A recipe defines required/optional roles, eligible asset tags, pose choices, support links, relative distances, orientation intervals, repetition limits and fallbacks. A style profile supplies recipe weights, palette preferences, occupancy and tidy/relaxed variation. These are proposed product controls, not universal rules of good design.

Choose groups and constrained anchors first, then sample a bounded set of positions and rotations. Reject hard failures before scoring valid candidates for spacing, balance, facing, repetition and empty space. Bounded backtracking or beam search can revise an earlier choice when a later item cannot fit.

Return a valid lower-density arrangement or an explicit incomplete-group reason when fitting fails. Do not silently distort assets or force an impossible count. Large lamps, books and small ornaments should each have explicit scale allowances.

10 Data work required for reliable placement

Asset preparation and library ownership

Asset preparation is part of the core product. A detailed mesh with an arbitrary pivot is not automatically ready for placement. Register each asset once, show its measured size and contact frame, and let the artist correct the data before it enters the generation pool.

Asset field	Why it is needed
Identity and provenance	Stable asset/version ID, source path, ownership/license record and variant family.
Physical dimensions	Known source units and dimensions at the approved scale.
Prepared poses	Upright/flat/leaning transforms, front axis and support assumptions.
Contacts	Base polygon or foot pads for each pose; optional centre of mass.
Collision shape	Box, capsule, convex or compound shape appropriate to the asset.
Child supports	Approved top region for a book, tray or other supporting asset.
Selection tags	Category, style/material/color metadata, recipe role and repetition limits.
Presentation	Thumbnail and original scene hierarchy/material references.

Derive a base estimate from bounds when useful, but require review for wide foliage, feet, hollow trays, asymmetric ornaments and leaning assets. Show the collider and contact pads in the viewport so inaccurate metadata can be corrected visibly.

Preserve the original asset hierarchy, renderer objects, materials and mapping. Generate native copies or instances through the Max adapter. Reconstructing a source from its extracted analysis triangles could lose its rendering behavior. Material/color tags should initially be authored; shader interpretation is renderer-specific.

Start with a small owned or commissioned accessory set and allow artists to register their own local assets. Commercial rendering permission does not automatically establish permission to redistribute meshes in a plugin library. Chaos explicitly makes Cosmos use subject to its EULA; the short commercial-use FAQ is insufficient to establish bundling rights. [S50, S51]

Register library files and proxy/texture dependencies with Max asset tracking rather than keeping important paths only in opaque settings. Autodesk documents asset enumeration and parameter-block file persistence. [S45, S46]

11 Proposed scene and library schema

Persistent data and deterministic regeneration

Record	Authoritative contents
FurnitureProfile	Furniture/category ID, source assembly, front/up frame and reusable analysis overrides.
SupportRegion	Host reference, local frame, boundary/holes, source association, clearance, accessibility, confidence and masks.
AssetDefinition	Source/version, units, poses, contacts, collision shape, child supports, tags and provenance.
RecipeDefinition	Roles, filters, support links, relative constraints, count/spacing rules, variation and fallbacks.
ScatterSetup	Native target/source references, recipe/style revision, seed, solver/schema version, overrides and locks.
PlacementResult	Asset/pose, support parent, local transform, group ID, ownership, accepted revision and lock state.

Store scene relationships through native references and parameter blocks. Max persistence and cloning provide contracts for referenced data and remapping. A JSON blob containing names, memory addresses or raw handles does not automatically gain those behaviors. Version the schema and keep accepted transforms so reopening an older scene does not rerun a changed solver. [S33, S35]

Derive independent random streams from setup, region and recipe-group identities. Sort inputs and resolve scoring ties deterministically. Worker completion order must not decide the winner. Replacing one vase or regenerating one compartment should not reshuffle the entire cabinet.

Detected regions do not have guaranteed identities after arbitrary topology changes. Match previous regions using local position, orientation, shape and source association, then flag ambiguous matches. Raw face indices alone are insufficient across reimport or remodeling.

Specify clone/merge policy: cloning furniture and its setup should remap the target; merging twice should resolve duplicate logical IDs; deleting a host should affect only explicitly owned output. Never find generated objects by a naming prefix and delete everything that matches.

Use real scene-node ownership for persistent metadata. Autodesk warns that AppData attached directly to the scene or root node is not saved. AppData may be a small metadata mirror, while native references own relationships. [S34]

Managed output should already exist as ordinary scene instances. Provide a bake/detach command for delivery, while preserving a separate editable setup. Plugin-free opening, rendering and farm behavior remain explicit tests; source renderers, textures and materials can still be dependencies.

12 A correction-friendly first UI

Artist workflow and diagnostics

The first user flow should be short: pick furniture, confirm category/front, analyze, inspect regions, choose a style/library, generate alternatives, lock useful groups and regenerate the rest. Reusable furniture profiles should reduce repeated setup for the same model.

Control or view	Purpose
Detected region overlay	Show usable, blocked and uncertain regions, with a legend and local clearance.
Region correction	Exclude, split, merge or author a region; mark closed compartments inaccessible.
Per-region overrides	Allowed categories, recipe, density and edge margin.
Alternatives and seed	Repeat an accepted arrangement and compare a few intentionally varied candidates.
Locks	Lock an asset, recipe group or compartment; preserve it as an obstacle.
Regenerate selection	Recompute only unlocked affected regions.
Manual edit policy	Offer explicit adopt, lock or detach behavior when generated output is moved.
Bake/detach	Deliver ordinary editable scene objects with a defined loss of regeneration.

Explain failures in terms an artist can act on: no eligible asset; opening too low; footprint too wide; collision with an existing object; region inaccessible; target changed while solving; or result would violate a lock. Include the relevant node or region and offer a viewport highlight.

An analysis report should summarize model dimensions, usable area, region count, minimum/variable clearances, questionable geometry and supported representation quality. Avoid displaying a confident success when the adapter analyzed only a display box.

Preserve negative space as a deliberate control. An occupancy target is an upper preference, not a promise that every shelf will be filled. Show when count, minimum gaps or real-world sizes make a request infeasible.

Keep existing scene props in the obstacle set by default within a defined scope. The user should be able to include/exclude an object explicitly. Existing TV bases, speakers and manually styled props should remain part of the geometric and functional context.

13 Candidate stack with a small initial footprint

C++ dependencies and distribution

Dependency	Proposed use	License finding
Clipper2	2D boundary union, difference, holes and margins.	Boost Software License 1.0 [S27].
Embree	Accelerated ray queries if benchmark results justify it.	Apache 2.0 [S28].
FCL	Collision/distance and broadphase candidate; evaluate overhead.	BSD; preserve applicable notices [S30].
CGAL	Useful algorithm reference or deliberately licensed option.	Package-dependent LGPL/GPL/commercial; relevant packages require scrutiny [S31].
Bullet	Optional later settling or physics checks.	Repository default zlib; inspect file-specific exceptions [S32].

Start with only what the first prototype needs. A compact planar extractor and simple colliders may be enough to validate the concept. Add ray and collision libraries behind interfaces so benchmark results can guide the choice rather than committing to every package researched.

Clipper2 uses integer coordinates internally, including scaled conversions for its double API. Choose a physical quantization/tolerance policy and local coordinates; extreme scaling is not a substitute for numerical design. [S26]

Pin dependency versions, compiler settings and transitive packages. FCL brings dependencies such as Eigen and libccd; their exact shipped versions and notices need review before packaging. Package licenses are not interchangeable with dataset or accessory rights. These findings are an engineering dependency record, not a legal opinion.

Build the geometry core as ordinary C++ and initially link it statically into each target-year Max adapter. This reduces an extra shared C++ ABI boundary. Keep allocations, exceptions and binary interfaces disciplined across any DLL boundaries that remain.

Package native binaries, MAXScript, icons and approved presets with Autodesk ApplicationPlugins and PackageContents.xml. Use release-specific component groups and tested version ranges. Do not assume that sharing v143 makes Max releases binary compatible; Max 2026 explicitly introduced an SDK break relative to 2025. [S40, S41]

A MAXScript-only interface can avoid adding managed UI dependencies. If a rich Qt UI is later required, match the Qt build expected by each Max target and use Autodesk integration guidance. [S42]

14 Verified baselines and unresolved setup details

SDK targets and the local development environment

Max target	Published C++ build guidance	Qt if a native UI is used
2025	SDK 2025; VS2022 17.8.3, v143; Windows SDK 10.0.19041.0 [S06].	6.5.3 [S06].
2026	SDK 2026; VS2022 17.8.3, v143; Windows SDK 10.0.19041.0 [S06].	6.5.3 [S06].
2027	Autodesk article: VS2022 17.14.13 IDE, v143/v14.38 build tools, C++20, Windows SDK 10.0.19041.0 [S07].	6.8.3 in the same article [S07].

The 2027 developer portal exists, but its SDK Requirements target could not be retrieved during this research. The 2027 row therefore uses an Autodesk-authored developer article. Broad Autodesk VFX alignment lists different Windows SDK values in some rows; use installed target SDK sample projects/property sheets and a successful sample build to settle exact pinning. [S07, S08]

Do not use settings from a USD-specific build document as the general plugin baseline when they conflict with the SDK-specific table. Separate target-year builds and actual load tests remain necessary. The thin MAXScript UI can be shared only after its commands and controls are validated on each target.

Local observation on this computer	What it establishes
Max 2026 executable: 28.3.2.30788	A read-only file-version check found this installed executable.
Max 2027 executable: 29.1.0.11426	A read-only file-version check found this installed executable.
Max 2024 installation directory present	Directory presence only; this report did not validate that application.
No SDK located in the checked installation-root paths	SDK installation is unconfirmed; this was not an exhaustive disk inventory.

Choose one installed release for the first bridge test. The user's preferred production version remains a product decision; installed folders do not settle it. First prove a sample builds, loads, exposes a command and returns correct measurements. Then expand compatibility.

15 What the first release should promise

Scope, difficult cases and risk controls

Risk	Effect	Control
Dirty or ambiguous furniture	Missed shelves or false usable surfaces.	Confidence overlays, authored overrides and a labelled fixture corpus.
Proxy mismatch	Analysis sees a box or incomplete shape.	Verified geometry adapters or separate analysis meshes.
Small or irregular geometry	Clearance sampling misses an obstacle.	Final 3D validation and tolerance-aware colliders.
Incorrect asset metadata	Floating, overhang, unstable stacks.	Contact/collider preview and registration checks.
Random but unattractive output	Artist cleanup erases time savings.	Authored recipes and close-up artist evaluation.
Scene state changes	Stale results, ownership errors or lost edits.	Revision checks, native references, locks and short commits.
Feature breadth	Whole-room scope delays a useful tool.	Rigid horizontal supports and a small asset set first.

Initial supported scope: static local furniture assemblies, known up/front, prepared rigid accessories, exposed tabletops/open shelves, approved book poses and simple stacks. Ordinary native meshes and authored analysis proxies should be first-class inputs.

Separate later work: pillows and soft furnishings, chairs pushed under tables, curtains, sloped/curved supports, arbitrary leaning stability, moving furniture, animated proxies, closed cabinets, door-swing/insertion simulation and fully automatic semantic recognition. These require different constraints and asset representations.

Closed cabinets illustrate a semantic limit: a hidden internal face can geometrically support an object but be irrelevant for styling. A furnished scene may also need screen, appliance, speaker or worktop zones that are not recoverable from mesh dimensions alone. Profiles and explicit markers can express those intentions.

A large-model performance claim is premature. Mesh evaluation, copying, acceleration-structure construction, solving and scene-node commit all need separate timing. A fast C++ solver cannot compensate for a long main-thread geometry extraction or thousands of slow node operations.

Document representation quality and supported cases precisely. Treat unsupported or uncertain geometry as a request for correction, rather than a silent approximate success.

16 Proposed implementation order

Development phases and validation gates

Phase	Concrete output	Gate before expanding scope
0. Platform bridge	Native sample, published commands, minimal rollout, units and measurement.	Loads on the chosen Max build; command works; no scene-state corruption.
1. Geometry inspector	Furniture assembly snapshot, usable-region overlay, clearance and diagnostics.	Compared with artist-labelled regions on varied shelves/tables; corrections persist.
2. Placement kernel	Prepared upright assets, contact/containment/collision checks and bounded search.	Known collision/support defects rejected; same inputs give the same result.
3. Recipe prototype	A few book/accessory groups, style controls and several alternatives.	Artists prefer useful results and spend less total time including cleanup.
4. Managed workflow	Save/load, clone/merge, locks, selective regeneration, undo and detach.	Lifecycle suite passes; ordinary output survives the intended delivery workflow.
5. Compatibility beta	Actual proxy adapters, large-scene profiling and target-year packages.	Measured performance and explicit per-renderer/per-year compatibility results.

Suggested initial content budget: roughly 20 varied furniture fixtures, 30-50 prepared accessories and 6-10 recipes. These are planning starting points, not research-derived minimum requirements. Include several sizes in each category so fitting tests do not depend on one convenient model.

The first engineering milestone should analyze one selected console or open shelf, display editable regions and produce one geometrically valid prepared arrangement. Include a difficult fixture early: separate boards, an internal shelf, a divider and an existing TV base.

Needed capabilities are a Max C++ developer, a geometry/placement developer, an archviz/content artist and QA coverage. Individuals may cover multiple roles. Asset preparation and artist evaluation must run alongside engine development.

Estimate a production schedule after measuring the bridge and region-extraction prototypes. The dominant uncertainty is generalization to real imported models and cleanup time, so a fixed delivery promise based only on algorithm documentation would be unreliable.

17 How to determine whether the product works

Benchmark plan and final decisions

Test family	Required fixtures or measurements
Geometry truth	Round/L-shaped tops, holes, bevels, thin boards, staggered shelves, protrusions, sloped ceilings and closed doors.
Import variants	Attached/separate parts, changed pivots, parent transforms, mirrors, nonuniform scales, inconsistent normals and unit changes.
Placement truth	Tall vases, wide foliage, narrow bases, multi-foot props, trays, stacks, existing clutter and fully contained colliders.
Lifecycle	Cancel; edit/delete while solving; save/reopen; clone; merge twice; undo/redo; reset/open; missing plugin; detach.
Performance	Snapshot/build/solve/commit time, peak memory and cold/cached results on named hardware.
Artist usefulness	Setup, correction and cleanup time; close-up render defects; blind preferences and variation quality.

For scale tests, use increasing input triangle counts such as 100k, 1m and 10m, and output counts such as 50, 500 and 5,000. These define a proposed benchmark envelope, not a performance promise. Report source extraction and commit bottlenecks separately from worker solve time.

Record region precision/recall or overlap against artist labels, accepted-placement defect rate, clearance errors, rejection reasons and lock preservation. Use the same assets/scenes to compare manual placement, Scatter Tools furniture mode and Bookmanager where their workflows overlap. Record total artist time rather than generation time alone.

Before implementation, settle the first Max target, primary renderer/proxy formats, local-asset registration workflow, category list, manual-edit behavior, controller-state ownership and distribution model. Reasonable defaults in this report allow a prototype, while those choices determine production scope.

Recommendation: proceed to a C++ geometry inspector and constrained recipe prototype with a thin MAXScript UI. Keep editable region detection, native scene output, real-scale assets and deterministic regeneration central. Expand beyond rigid horizontal supports only after fixtures and artist trials demonstrate reliable results.

Research establishes that the architecture is supported and that directly relevant algorithms and products exist. It does not establish a working implementation, measured speed, renderer compatibility or universal furniture understanding. Those are the explicit outputs of the proposed validation phases.

Primary sources

Vendor manuals, Autodesk documentation, author projects and package terms. Links are clickable. Documentation records capabilities and constraints; proposed algorithms and product decisions are identified in the report. Accessed 17 October 2026.

S01 Autodesk Function Publishing
https://help.autodesk.com/cloudhelp/2021/ENU/Max-Developer-Help/developer/3ds_max_sdk_features/function_publishing.html
C++ interfaces callable from MAXScript.

S02 Autodesk Thread Safety
https://help.autodesk.com/cloudhelp/2025/JPN/MAXDEV-Developer/files/best_practices/thread_safety.html
Reference and node evaluation systems are single threaded; the page body is English.

S03 Autodesk Extracting the Mesh from a Node
https://help.autodesk.com/cloudhelp/2025/ENU/MAXDEV-Developer/files/3ds_max_sdk_features/modeling/meshes/extracting_the_mesh_from_a_node.html
Evaluated geometry, conversion, and temporary ownership.

S04 Autodesk Object Transformation Matrix
https://help.autodesk.com/cloudhelp/2025/ENU/MAXDEV-Developer/files/3ds_max_sdk_features/modeling/transformation_and_rotation/object_transformation_matrix.html
Object offset and world-space modifier coordinate considerations.

S05 Autodesk units.h API
https://help.autodesk.com/cloudhelp/2025/ENU/MAXDEV-CPP-API-REF/units_8h.html
GetSystemUnitScale and GetSystemUnitInfo.

S06 Autodesk SDK Requirements for 2025 and 2026
https://help.autodesk.com/cloudhelp/2026/ENU/MAXDEV-Developer/files/about_the_3ds_max_sdk/sdk_requirements.html
SDK, compiler, Windows SDK, and Qt baselines.

S07 Autodesk Developing for Max 2027 in Visual Studio 2026
https://blog.autodesk.io/setting-up-3ds-max-2027-sdk-development-with-visual-studio-2026/
Autodesk developer article dated 31 July 2026; 2027 compiler and Qt detail.

S08 Autodesk VFX Platform Alignment
https://aps.autodesk.com/developer/overview/me-vfx-platform-alignment
Broad platform matrix; some Windows SDK entries differ from SDK-specific sources.

S09 Spline Dynamics Scatter Tools
https://www.splinedynamics.com/randomixer-scatter-tools/
Furniture population, redistribution of existing objects, and script-based workflow.

S10 Spline Dynamics Scatter Tools Manual
https://www.splinedynamics.com/randomixer-scatter-tools-manual/
Horizontal furniture surfaces, anchors, collision claims, and failure behavior.

S11 model+model Bookmanager 2
https://www.modelplusmodel.com/plugins/bookmanager.html
Shelf polygons, prepared book orientations, seeds, and pivot correction.

S12 Chaos Scatter Scattering Rollout
https://docs.chaos.com/display/VMAX/Scattering%2BRollout
Distribution controls and approximate instance overlap handling.

S13 ForestPack Surfaces
https://docs.itoosoft.com/forestpack/forest-plugin/surfaces
Surface projection, multiple surfaces, filtering, and refresh behavior.

S14 RailClone Bookshelf Tutorial
https://www.itoosoft.com/tutorials/bookshelf-books
Procedural construction and arrangement of shelves and books.

S15 SiNi Disperse Overview
https://docs.sinisoftware.com/disperse-overview
Artist-painted geometry and proxy placement.

S16 HSM Research Paper
https://arxiv.org/html/2503.16848v3
Support extraction, compartments, hierarchy, motifs, and limitations.

S17 HSM Official Project
https://3dlg-hcvc.github.io/hsm/
Hierarchical Scene Motifs, presented at 3DV 2026.

S18 SceneMotifCoder Official Project
https://3dlg-hcvc.github.io/smc/
Example-driven parameterized programs for recurring arrangements.

S19 SceneSmith Research Paper
https://arxiv.org/html/2602.09153v1
Hierarchical planning and geometric/physical refinement.

S20 Make It Home Research Paper
https://ww3.math.ucla.edu/camreport/cam11-49.pdf
Spatial priors and ergonomic objectives for furniture layout.

S21 Clutterpalette Research Paper
https://www.cs.umb.edu/~craigyu/papers/clutterpalette.pdf
Support-aware semantic suggestions for small objects.

S22 PlanIT Author Repository
https://github.com/brownvc/planit
Graph planning and instantiation; repository warns of unavailable models/metadata.

S23 CGAL Shape Detection
https://doc.cgal.org/latest/Shape_detection/index.html
Planar region growing and fitting tolerances.

S24 CGAL Side of Triangle Mesh
https://doc.cgal.org/5.6.3/Polygon_mesh_processing/classCGAL_1_1Side__of__triangle__mesh.html
Closed mesh and self-intersection preconditions for containment.

S25 Generalized Winding Numbers Author Project
https://igl.ethz.ch/projects/winding-number/
Inside/outside reasoning for imperfect meshes and its assumptions.

S26 Clipper2 Overview
https://angusj.com/clipper2/Docs/Overview.htm
Polygon boolean operations, offsets, holes, and integer scaling.

S27 Clipper2 License
https://github.com/AngusJohnson/Clipper2/blob/main/LICENSE
Boost Software License 1.0.

S28 Embree Official Project
https://www.embree.org/
Ray-query acceleration and Apache 2.0 license.

S29 Flexible Collision Library
https://github.com/flexible-collision-library/fcl
Collision/distance queries, broadphase, and dependencies.

S30 FCL License
https://github.com/flexible-collision-library/fcl/blob/master/LICENSE
BSD notice conditions.

S31 CGAL Licensing
https://www.cgal.org/license.html
Package-dependent LGPL/GPL and commercial licensing.

S32 Bullet Official Repository
https://github.com/bulletphysics/bullet3
Optional dynamics; default zlib license with file-specific exceptions.

S33 Autodesk Loading and Saving Plugin Data
https://help.autodesk.com/cloudhelp/2026/ENU/MAXDEV-Developer/files/writing_plug-ins/plug-in_basics/loading_and_saving_plug-in_data.html
Referenced data persistence and explicit save/load responsibilities.

S34 Autodesk Application Data
https://help.autodesk.com/cloudhelp/2024/ENU/Max-Developer-Help/3ds_max_sdk_features/scenes_and_nodes/custom_scene_data/application_data.html
Node AppData; warning about scene/root-node ownership.

S35 Autodesk ReferenceTarget API
https://help.autodesk.com/cloudhelp/2025/ENU/MAXDEV-CPP-API-REF/class_reference_target.html
Clone and reference remapping contracts.

S36 Autodesk Post Load Callbacks
https://help.autodesk.com/cloudhelp/2024/ENU/Max-Developer-Help/3ds_max_sdk_features/miscellaneous_topics/post-load_callbacks.html
References and safe ordering of post-load work.

S37 Autodesk Undo and Redo Integration
https://help.autodesk.com/cloudhelp/2023/ENU/Max-Developer-Help/3ds_max_sdk_features/the_undo_and_redo_system/how_to_enable_undo_and_redo.html
Hold/restore rules, including caller-owned Function Publishing transactions.

S38 Autodesk Hold API
https://help.autodesk.com/cloudhelp/2026/ENU/MAXDEV-CPP-API-REF/class_hold.html
Matching transaction begin/accept/cancel and parameter-block undo.

S39 Autodesk Scene Event Manager
https://help.autodesk.com/cloudhelp/2023/ENU/Max-Developer-Help/cpp_ref/class_i_scene_event_manager.html
Batched and delayed scene-change notifications.

S40 Autodesk Packaging Plugins
https://help.autodesk.com/cloudhelp/2026/ENU/MAXDEV-Developer/files/writing_plug-ins/plugin_package/packaging_plugins.html
ApplicationPlugins, PackageContents.xml, and release-specific components.

S41 Autodesk Max 2026 SDK Changes
https://help.autodesk.com/cloudhelp/2026/ENU/MAXDEV-Developer/files/what_s_new/whats_new_3dsmax_2026_sdk.html
SDK break and runtime changes.

S42 Autodesk Qt Plugin Integration
https://help.autodesk.com/cloudhelp/2026/ENU/MAXDEV-Developer/files/writing_plug-ins/using_qt_with_3ds_max_plug-ins.html
Optional native C++ Qt UI and version integration.

S43 Chaos VRayProxy
https://docs-chaos.atlassian.net/wiki/spaces/VMAX/pages/113575423
Preview/full mesh distinctions, mesh import, and proxy transforms.

S44 Chaos Corona Proxy
https://docs.chaos.com/display/CRMAX/Corona%2BProxy
Bounding-box, point-cloud, and full-mesh display modes.

S45 Autodesk Enumerating and Exposing Assets
https://help.autodesk.com/cloudhelp/2024/ENU/Max-Developer-Help/3ds_max_sdk_features/scenes_and_nodes/assets/enumerating_and_exposing_assets.html
EnumAuxFiles and external scene dependencies.

S46 Autodesk Saving and Loading Assets
https://help.autodesk.com/cloudhelp/2022/ENU/Max-Developer-Help/3ds_max_sdk_features/scenes_and_nodes/assets/saving_and_loading_assets.html
Parameter-block asset paths and automatic persistence.

S47 HSM Code License
https://github.com/3dlg-hcvc/hsm/blob/main/LICENSE.txt
MIT software license.

S48 HSM Dataset Terms
https://huggingface.co/datasets/3dlg-hcvc/hsm
CC BY-NC 4.0; separate from the code license.

S49 HSSD Models Dataset Terms
https://huggingface.co/datasets/hssd/hssd-models
CC BY-NC 4.0 model dataset.

S50 Chaos Cosmos Commercial Use FAQ
https://support.chaos.com/hc/en-us/articles/4403292893329-Can-I-use-Cosmos-assets-commercially
Commercial use remains subject to the applicable EULA.

S51 Chaos EULA
https://www.chaos.com/legal/eula
Product and content terms; asset redistribution must be separately established.

S52 Autodesk Plugin Base Classes
https://help.autodesk.com/cloudhelp/2023/ENU/Max-Developer-Help/3ds_max_sdk_features/plug-ins/plug-in_base_classes.html
GUP, utility, helper, and other native plugin roles.